

Chapter5: Data Compression

Learning Objectives

After completing this chapter, students will be able to:

- Define data compression and its importance.
 - Explain the fundamentals of compression.
 - Understand the relationship between storage space and compression.
 - Describe the need for coding in data compression.
 - Differentiate between lossless and lossy compression techniques.
 - Compare various compression algorithms and their applications.
-

1. Introduction to Data Compression

Definition

Data Compression is the process of reducing the size of digital data by eliminating redundancy and representing information more efficiently. The objective is to save storage space, reduce transmission time, lower bandwidth requirements, and improve overall system performance.

Data compression is a fundamental component of **multimedia computing**, as multimedia files such as images, audio, and videos require large amounts of storage and network bandwidth.

Why Data Compression is Important

Modern multimedia systems generate enormous amounts of data.

Examples include:

- High-resolution photographs
- HD and 4K videos
- Digital audio recordings
- Medical images
- Satellite images
- Surveillance videos

Without compression, storing and transmitting such files would be costly and inefficient.

Applications of Data Compression

Data compression is widely used in:

- Digital photography
- Video streaming (YouTube, Netflix)
- Audio streaming (Spotify, Apple Music)
- Video conferencing
- Medical imaging
- Cloud storage
- Mobile communication
- Database backup
- Artificial Intelligence datasets

Data Compression

Introduction

Data Compression is the process of reducing the size of digital data by representing it using fewer bits than the original. The primary objective is to save storage space and reduce the time required to transmit data over communication networks.

Compression works by identifying and eliminating redundant or unnecessary information in data. After compression, the data occupies less space and can be stored or transmitted more efficiently. When needed, the compressed data is restored through a process called **decompression**.

Example

Suppose a text file contains the following data:

AAAAAABBBBBBCCCCC

Instead of storing each character individually, a compression algorithm can represent it as:

6A5B6C

This compressed representation requires fewer bits, thereby saving storage space.

Why Data Compression is Important

Data compression is widely used because digital data such as images, videos, audio, documents, and software continues to grow rapidly. Compression helps in:

- Saving storage space
- Increasing transmission speed
- Reducing communication costs
- Improving system performance
- Making multimedia applications practical

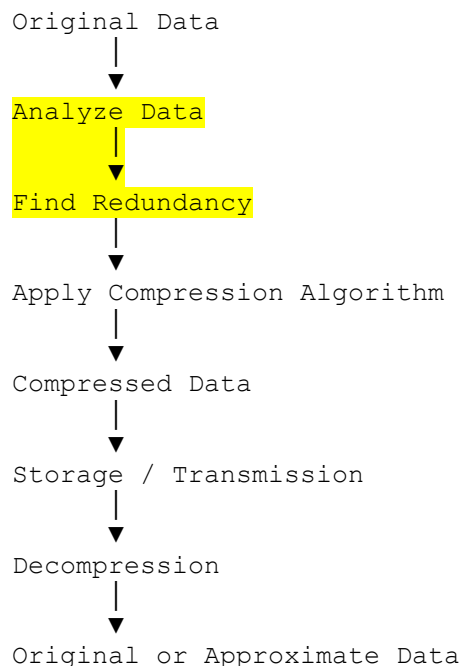
Examples include:

- ZIP files for documents
- JPEG for images
- MP3 for audio
- MP4 (H.264/H.265) for videos

Basics of Compression

The basic idea behind compression is to **remove redundancy** or represent data more efficiently without affecting usability.

Basic Compression Process



How Compression Works

Step 1: Input Data

The system receives the original file (text, image, audio, or video).

Example:

AAAAABBBBBBBBBCCCCDDDD

Step 2: Data Analysis

The compression algorithm analyzes the data to detect:

- Repeated symbols
 - Frequent patterns
 - Similar values
 - Statistical redundancy
-

Step 3: Remove Redundancy

Repeated information is encoded more efficiently.

Example:

Instead of storing:

AAAAA

Store:

5A

Step 4: Compression

The selected compression algorithm encodes the data into a compact form.

Examples of algorithms:

- Run-Length Encoding (RLE)
- Huffman Coding
- Lempel-Ziv (LZ77/LZ78)

- JPEG Compression
- MPEG Compression

Step 5: Compressed File

The compressed file occupies less storage space.

Example:

Original file:

100 MB

Compressed file:

20 MB

Step 6: Decompression

The receiver decompresses the file to recover the original data (lossless) or an approximation (lossy).

2. Basics of Compression

Compression works by reducing redundancy in data.

There are three main types of redundancy:

A. Spatial Redundancy

Occurs when neighboring pixels have similar values.

Example:

```
255 255 255 255
255 255 255 255
255 255 255 255
```

Instead of storing every value separately, repeated values can be represented more efficiently.

B. Temporal Redundancy

Occurs when consecutive video frames are very similar.

Example:

Frame 1

Person Standing

Frame 2

Person Standing (Almost Same)

Only the changes between frames need to be stored.

Used in:

- MPEG
- H.264
- H.265
- AV1

C. Coding Redundancy

Occurs when longer binary codes are assigned to frequently occurring symbols.

Efficient coding assigns shorter codes to more frequent symbols.

Example:

Letter Frequency

Character Frequency

A	Very High
B	Medium
Z	Very Low

Compression algorithms such as **Huffman Coding** use variable-length codes to reduce the average number of bits required.

Storage Space

What is Storage Space?

Storage space refers to the amount of digital memory required to store data in a storage device such as a hard disk, SSD, USB drive, memory card, or cloud storage.

Storage capacity is measured in:

Unit	Size
Bit (b)	Smallest unit of data
Byte (B)	8 bits
Kilobyte (KB)	1,024 Bytes
Megabyte (MB)	1,024 KB
Gigabyte (GB)	1,024 MB
Terabyte (TB)	1,024 GB

Why Saving Storage Space is Important

Modern applications generate huge amounts of data, such as:

- High-resolution images
- 4K and 8K videos
- Audio recordings
- Databases
- Scientific data
- Medical imaging

Without compression, storage devices would fill up quickly.

Example

Suppose a camera captures:

- 100 images
- Each image = 10 MB

Total storage required:

$$100 \times 10 \text{ MB} = 1000 \text{ MB} = 1 \text{ GB}$$

After JPEG compression:

Each image = 2 MB

Total storage:

$$100 \times 2 \text{ MB} = 200 \text{ MB}$$

Storage saved = 800 MB (80%)

Benefits of Saving Storage Space

- More files can be stored on the same device.
 - Lower storage costs.
 - Faster file backup and restoration.
 - Better cloud storage utilization.
 - Easier data management.
-

Need of Coding

What is Coding?

In data compression, **coding** is the process of assigning binary codes (0s and 1s) to symbols or data values in a way that reduces the total number of bits required for storage or transmission.

Efficient coding forms the basis of many compression algorithms.

Why is Coding Needed?

Without coding, every character or symbol may require the same number of bits.

Example (ASCII):

Character	Binary Code
A	01000001
B	01000010
C	01000011

Each character uses **8 bits**, regardless of how frequently it appears.

If some characters appear much more often than others, using fixed-length codes is inefficient.

3. Storage Space

Definition

Storage space is the amount of memory required to store digital information.

The size of multimedia files depends on:

- Resolution
- Color depth
- Sampling rate
- Bit depth
- Duration
- Compression method

Mathematical Formula

Image Storage

Image:

- Resolution = 1920×1080
- Color Depth = 24 bits

Storage

49,766,400 bits

Convert to bytes

6,220,800 \text{ bytes}
 ≈ 5.93 MB

Compression Ratio

Compression effectiveness is measured by:

Example

Original File = 100 MB

Compressed File = 25 MB

Meaning the compressed file is four times smaller than the original.

4. Need for Coding

Definition

Coding is the process of representing data using binary symbols efficiently.

Instead of assigning fixed-length binary codes, variable-length coding minimizes storage requirements.

Why Coding is Necessary

- Reduces file size.
 - Increases transmission speed.
 - Saves storage.
 - Improves communication efficiency.
 - Reduces network bandwidth usage.
-

Types of Coding

Fixed-Length Coding

Every symbol uses the same number of bits.

Example

Character Code

A 000

B	001
C	010
D	011

Simple but not always efficient.

Variable-Length Coding

Frequently occurring symbols receive shorter binary codes.

Example

Character Code	
A	0
B	10
C	110
D	111

This approach significantly reduces the average number of bits required.

Popular Coding Techniques

Huffman Coding

Assigns shorter codes to frequently occurring symbols.

Advantages:

- Optimal prefix code
- Lossless
- Widely used

Applications:

- ZIP
 - PNG
 - JPEG (entropy coding stage)
-

Arithmetic Coding

Represents an entire message as a single fractional number.

Advantages:

- Better compression than Huffman in many cases
- Efficient for complex probability distributions

Applications:

- JPEG2000
- H.264
- HEVC

Run Length Encoding (RLE)

Stores repeated values as:

Value + Count

Example

Original

AAAAAABBBBBCCCC

Compressed

A7B5C4

Best suited for images with large areas of identical colors.

Lossy and Lossless Compression Techniques

Data compression is the process of reducing the size of digital data so that it requires less storage space and can be transmitted more efficiently. Compression techniques are broadly classified into **Lossless Compression** and **Lossy Compression**.

1. Lossless Compression

Definition

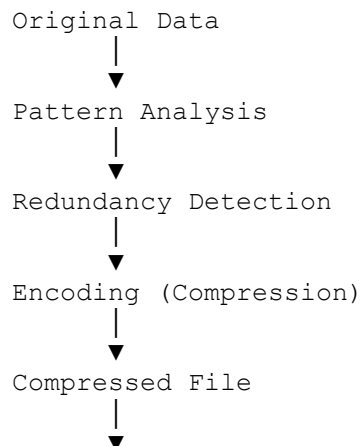
Lossless compression is a compression technique in which **all the original data can be perfectly reconstructed** after decompression. No information is lost during the compression process.

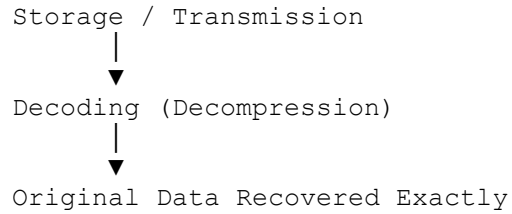
It is used when maintaining the exact original data is essential, such as in text documents, program files, databases, and medical images.

Key Characteristics

- Original data is completely recovered.
 - No loss of quality.
 - Suitable for text and important documents.
 - Lower compression ratio compared to lossy compression.
 - Data integrity is maintained.
-

Working Principle (Process Stages)





Stage-by-Stage Explanation

Stage 1: Original Data

The compression process begins with the original file.

Example:

AAAAABBBBBBBBBCCCCDDDD

Stage 2: Pattern Analysis

The algorithm searches for repeated patterns, duplicate symbols, or statistical redundancies.

Example:

AAAAA
BBBBB
CCCCC

Repeated characters are identified.

Stage 3: Redundancy Detection

Repeated information is recognized.

Example

Instead of storing

AAAAA

the system stores

5A

Stage 4: Encoding

The repeated information is encoded using a compression algorithm.

Common algorithms include

- Run-Length Encoding (RLE)
 - Huffman Coding
 - Arithmetic Coding
 - Lempel-Ziv (LZ77, LZ78)
 - DEFLATE (ZIP)
-

Stage 5: Compressed File

The encoded data occupies less storage space.

Example

Original

AAAAABBBBBCCCCCDDDD

Compressed

5A5B5C4D

Stage 6: Decompression

The decoder reverses the encoding.

5A

becomes

AAAAA

Stage 7: Original File Restored

The decompressed file is **100% identical** to the original.

Original File = Decompressed File

No information is lost.

Example of Lossless Compression

Original Data

AAAAABBBBCCCCDD

Compression

5A4B4C2D

Decompression

AAAAABBBBCCCCDD

Exactly the same as the original.

Common Lossless Compression Algorithms

1. Run-Length Encoding (RLE)

Compresses repeated characters.

Example

AAAAAA

↓

6A

Used in

- Bitmap images
 - Fax machines
-

2. Huffman Coding

Assigns shorter binary codes to frequently occurring symbols.

Example

A → 0
B → 10
C → 110
D → 111

Frequently used in

- ZIP
- PNG
- PDF

3. Lempel-Ziv (LZ77/LZ78)

Stores repeated strings using dictionary references.

Used in

- ZIP
- GIF
- PNG

Applications of Lossless Compression

- ZIP files
- Text documents
- Executable programs
- Database backup
- Medical images
- Legal documents
- Source code
- Financial records

Advantages

- No data loss
- Perfect reconstruction
- High accuracy
- Suitable for sensitive data

- Maintains original quality
-

Disadvantages

- Lower compression ratio
 - Larger compressed files than lossy methods
 - Slower for very large multimedia files
-

2. Lossy Compression

Definition

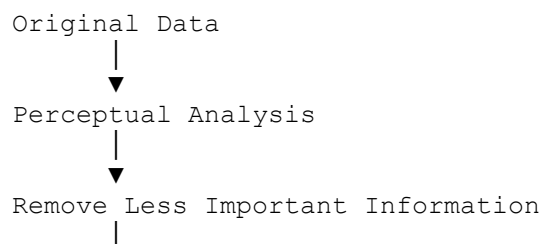
Lossy compression permanently removes some information that is considered less important or less noticeable to human perception. After decompression, the recovered data is **not identical** to the original.

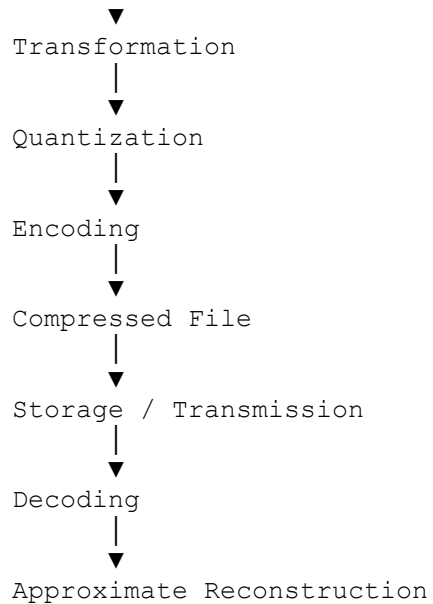
Lossy compression achieves much higher compression ratios than lossless compression.

Key Characteristics

- Some information is permanently removed.
 - High compression ratio.
 - Smaller file size.
 - Slight reduction in quality.
 - Original file cannot be perfectly recovered.
-

Working Principle (Process Stages)





Stage-by-Stage Explanation

Stage 1: Original Data

Original image, audio, or video enters the compression system.

Stage 2: Perceptual Analysis

The compressor identifies information that humans are less likely to notice.

Examples

For images:

- Tiny color differences
- Very fine details

For audio:

- Sounds outside the human hearing range
 - Masked frequencies
-

Stage 3: Remove Less Important Information

The unnecessary details are discarded permanently.

Example

Original image

16 million colors

Compressed image

4 million colors

Stage 4: Transformation

The image or audio is transformed into another mathematical domain.

Common transforms

- Discrete Cosine Transform (DCT)
- Discrete Wavelet Transform (DWT)
- Fourier Transform

These transforms separate important information from less important details.

Stage 5: Quantization

Quantization removes small coefficients that contribute little to perceived quality.

This is the **main source of information loss**.

Stage 6: Encoding

The remaining information is encoded efficiently using techniques such as Huffman coding or arithmetic coding.

Stage 7: Compressed File

A much smaller file is generated.

Example

Original Image = 20 MB

Compressed Image = 2 MB

Stage 8: Decompression

The decoder reconstructs an approximation of the original.

Some details cannot be restored because they were permanently removed.

Example of Lossy Compression

Original image

1920 × 1080
24-bit color

Compressed JPEG

Same resolution

↓

Reduced color precision

↓

Smaller file size

The image looks almost identical to the human eye, but some fine details are lost.

Common Lossy Compression Algorithms

JPEG

Used for

- Digital photographs

- Websites
- Cameras

Uses

- DCT
 - Quantization
 - Huffman Coding
-

MP3

Used for

- Music
- Audio streaming

Removes sounds outside human hearing capability.

AAC

Improved version of MP3.

Used in

- YouTube
 - Apple Music
 - iTunes
-

MPEG / H.264 / H.265

Used for

- Movies
- Video streaming
- Television
- YouTube
- Netflix

Compresses by removing redundant information between consecutive video frames.

Applications of Lossy Compression

- Digital photography
- Video streaming
- Online music
- YouTube
- Netflix
- Video conferencing
- Mobile multimedia
- Social media

Advantages

- Very high compression ratio
- Much smaller file size
- Faster transmission
- Saves storage space
- Ideal for multimedia

Disadvantages

- Permanent information loss
- Reduced quality after compression
- Multiple recompressions may introduce visible artifacts
- Not suitable for text or important documents

Comparison Between Lossless and Lossy Compression

Feature	Lossless Compression	Lossy Compression
Data Loss	No	Yes
Original Recovery	Exact	Approximate

Feature	Lossless Compression	Lossy Compression
Quality	Unchanged	Reduced
Compression Ratio	Low to Moderate	High
File Size	Larger	Smaller
Suitable For	Text, software, medical data	Images, audio, video
Repeated Compression	No quality degradation	Quality degrades over time
Common Formats	ZIP, PNG, GIF, FLAC	JPEG, MP3, AAC, MPEG, H.264

Real-World Examples

Data Type	Lossless Format	Lossy Format
Images	PNG, GIF	JPEG, WebP (lossy mode)
Audio	FLAC, WAV (uncompressed)	MP3, AAC, OGG
Video	FFV1, HuffYUV	MPEG-4, H.264, H.265, AV1 (lossy mode)
Documents	ZIP, PDF (lossless compression for text)	Not commonly applicable

Summary

- **Lossless compression** preserves all original information, making it ideal for text, software, databases, and medical records where accuracy is essential. It identifies and encodes redundant data without discarding any information.
- **Lossy compression** achieves much higher compression by removing less perceptible information based on human visual or auditory characteristics. It is widely used for images, audio, and video, where small quality losses are acceptable in exchange for significantly reduced file sizes.

5. Lossless Compression

Definition

Lossless compression reduces file size **without losing any original information**.

After decompression, the recovered data is **identical** to the original.

Characteristics

- Original data is perfectly restored.
 - No quality loss.
 - Suitable for documents and critical data.
-

Common Algorithms

- Huffman Coding
 - Run Length Encoding (RLE)
 - LZW (Lempel–Ziv–Welch)
 - DEFLATE
 - Arithmetic Coding
-

File Formats Using Lossless Compression

- PNG
 - GIF
 - TIFF (optional)
 - ZIP
 - FLAC (audio)
-

Advantages

- Perfect reconstruction.
 - High accuracy.
 - Suitable for medical and scientific applications.
-

Disadvantages

- Lower compression ratio compared to lossy methods.
 - Larger compressed file sizes.
-

6. Lossy Compression

Definition

Lossy compression permanently removes less important information to achieve much higher compression ratios.

The original data cannot be perfectly reconstructed.

Characteristics

- Some information is discarded.
 - Smaller file sizes.
 - Slight quality reduction.
 - Often imperceptible to the human eye or ear.
-

Popular Algorithms

- JPEG
 - MPEG
 - MP3
 - AAC
 - H.264
 - H.265 (HEVC)
 - AV1
-

Advantages

- Very high compression ratio.
 - Faster transmission.
 - Reduced storage costs.
-

Disadvantages

- Permanent loss of information.

- Repeated compression may further degrade quality.

Comparison: Lossless vs. Lossy Compression

Feature	Lossless Compression	Lossy Compression
Data Recovery	Exact	Approximate
Quality	No loss	Some loss
Compression Ratio	Lower	Higher
File Size	Larger	Smaller
Repeated Editing	No quality degradation	Quality may degrade
Typical Applications	Documents, medical images, archives	Photos, audio, video streaming
Examples	PNG, ZIP, FLAC	JPEG, MP3, MPEG, H.265

Real-World Examples

Application	Compression Technique
Medical Imaging	Lossless
Digital Documents	Lossless
Banking Records	Lossless
Software Archives	Lossless
Digital Photography	Lossy (JPEG)
Video Streaming	Lossy (H.264, H.265, AV1)
Music Streaming	Lossy (MP3, AAC)
Social Media Images	Mostly Lossy

Advantages of Data Compression

- Saves storage space.
 - Reduces bandwidth requirements.
 - Speeds up file transmission.
 - Lowers cloud storage costs.
 - Improves multimedia performance.
 - Enables efficient backup and archiving.
-

Limitations of Data Compression

- Compression and decompression require computational resources.
 - Lossy compression may reduce quality.
 - Different applications require different compression methods.
 - Very high compression ratios can introduce noticeable artifacts.
-

Summary

Data compression is an essential technology in multimedia computing that reduces file size by removing redundancy and encoding information efficiently. It helps save storage space, decreases transmission time, and improves network performance. **Lossless compression** preserves all original data and is ideal for documents, medical images, and archives, while **lossy compression** achieves much higher compression ratios by discarding less important information, making it suitable for images, audio, and video. Understanding the principles of coding, compression ratio, and storage optimization is fundamental for designing efficient multimedia systems.

Source Coding, Entropy Coding, Hybrid Coding, Lossy Sequential DCT-Based Mode, Expanded Lossy DCT-Based Mode, JPEG and MPEG Compression

Learning Objectives

After studying this chapter, students will be able to:

- Understand the concepts of source coding, entropy coding, and hybrid coding.
 - Explain the principles of Discrete Cosine Transform (DCT)-based image compression.
 - Describe the Sequential DCT-Based Mode and Expanded DCT-Based Mode in JPEG.
 - Understand the JPEG image compression process.
 - Explain MPEG video compression techniques.
-

1. Source Coding

Introduction

Source coding is the process of representing information from a source in the most efficient way by removing redundancy while preserving the required information. It is also known as **data compression coding**.

The primary objectives of source coding are:

- Reduce storage requirements.
 - Reduce transmission bandwidth.
 - Improve communication efficiency.
 - Minimize transmission time.
 - Reduce storage costs.
-

Basic Principle

Suppose a source generates the message:

AAAAABBBBCCCCCDDDDD

Instead of storing every character individually, source coding compresses the message using efficient coding techniques such as:

A5 B4 C6 D5

This reduces the number of bits required.

Types of Source Coding

A. Lossless Source Coding

The original data can be reconstructed exactly.

Examples:

- Huffman Coding
- Run Length Encoding (RLE)
- LZW
- Arithmetic Coding

Applications

- Text documents
 - Medical images
 - Software archives
-

B. Lossy Source Coding

Some information is discarded to achieve higher compression.

Examples

- JPEG
- MPEG
- MP3
- H.264
- HEVC

Applications

- Digital photography
 - Video streaming
 - Music compression
-

Advantages

- Saves storage space.
 - Reduces bandwidth.
 - Faster transmission.
 - Improves multimedia performance.
-

2. Entropy Coding

Definition

Entropy coding is a **lossless compression technique** that assigns **short binary codes** to **frequently occurring symbols** and **longer codes** to **less frequent symbols**.

It removes **coding redundancy** without losing information.

Entropy coding is based on **Claude Shannon's Information Theory**.

Entropy

Entropy measures the **average amount of information** produced by a source and represents the theoretical minimum number of bits required to encode each symbol.

Mathematically,

Higher entropy means the data is more random and harder to compress.

Lower entropy means there is more redundancy and better compression is possible.

Example

Suppose

Symbol Probability

A	0.50
B	0.25
C	0.15
D	0.10

Possible Huffman codes

Symbol Huffman Code

A	0
B	10
C	110
D	111

Frequently occurring symbols receive shorter codes.

Popular Entropy Coding Techniques

Huffman Coding

- Variable-length coding
- Prefix-free codes
- Fast decoding
- Used in JPEG and ZIP files

Huffman Coding

Introduction

Huffman Coding is a **lossless data compression algorithm** developed by **David A. Huffman** in 1952. It is one of the most widely used **entropy coding** techniques in multimedia compression because it reduces the average number of bits required to represent data **without losing any information**.

Huffman coding is based on a simple principle:

Frequently occurring symbols are assigned shorter binary codes, while less frequently occurring symbols are assigned longer binary codes.

This minimizes the average code length and achieves efficient compression.

Why Huffman Coding is Needed

Suppose we want to store the following message:

AAAAABBC

Using **ASCII encoding**, each character requires **8 bits**.

There are **8 characters**, so the total storage required is:

However, the character **A** appears much more frequently than the others. Using the same number of bits for every character is inefficient. Huffman coding reduces the storage requirement by assigning shorter codes to more frequent characters.

Basic Principle

1. Count the frequency of each symbol.
2. Build a binary tree (Huffman Tree).

3. Assign binary codes:
 - Left branch = **0**
 - Right branch = **1**
4. Replace each symbol with its Huffman code.

The resulting codes are **prefix-free**, meaning no code is the prefix of another. This ensures unique decoding.

Algorithm Steps

Step 1: Count Symbol Frequencies

Consider the message:

ABBCCDDDD

The frequencies are:

Symbol Frequency

A	1
B	2
C	3
D	4

Step 2: Arrange Frequencies

Sort the symbols from the smallest frequency to the largest.

A = 1, B = 2, C = 3, D = 4

Step 3: Build the Huffman Tree

Combine the two nodes with the smallest frequencies repeatedly.

Iteration 1

Combine:

A (1) + B (2)

Result:

AB (3)

Remaining nodes:

AB (3)

C (3)

D (4)

Iteration 2

Combine:

AB (3) + C (3)

Result:

ABC (6)

Remaining:

D (4)

ABC (6)

Iteration 3

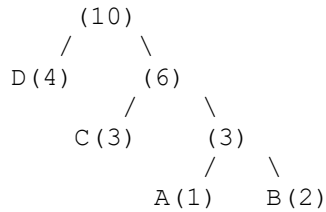
Combine:

D (4) + ABC (6)

Result:

Root (10)

Huffman Tree



Assign:

- Left branch = 0
- Right branch = 1

Step 4: Generate Huffman Codes

Trace the path from the root to each symbol.

Symbol	Path	Huffman Code
D	Left	0
C	Right → Left	10
A	Right → Right → Left	110
B	Right → Right → Right	111

Step 5: Encode the Message

Original message:

ABBCCDDDD

Replace each symbol with its Huffman code.

Symbol	Code
A	110
B	111
B	111

C	10
C	10
C	10
D	0
D	0
D	0
D	0

Encoded bitstream:

1101111111010100000

Storage Comparison

Before Compression (Fixed-Length Coding)

There are **4 unique symbols**, so we need **2 bits per symbol**.

Symbol Fixed Code

A	00
B	01
C	10
D	11

Message:

ABBCCDDDD

Arithmetic Coding

Instead of assigning codes to individual symbols, arithmetic coding represents the **entire message** as a single fractional number.

Advantages

- Higher compression efficiency
- Better than Huffman for complex data

Applications

- JPEG2000
 - H.264
 - HEVC
-

Advantages

- No information loss.
 - Efficient coding.
 - High compression efficiency.
 - Simple decoding.
-

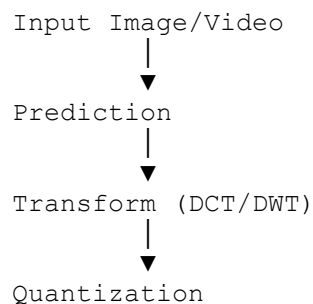
3. Hybrid Coding

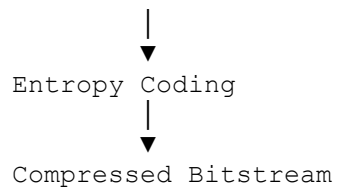
Definition

Hybrid coding combines **multiple compression techniques** to obtain higher compression efficiency than using a single method.

Modern multimedia compression standards use hybrid coding because different algorithms are effective at reducing different types of redundancy.

Hybrid Coding Process





Components

Prediction Coding

Predicts pixel values using neighboring pixels.

Removes spatial redundancy.

Transform Coding

Converts image information from the spatial domain into the frequency domain.

Common transforms:

- DCT
 - DWT
-

Quantization

Removes visually insignificant information.

This is the only **lossy** stage in JPEG compression.

Entropy Coding

Compresses the remaining information using:

- Huffman Coding
 - Arithmetic Coding
-

Advantages

- Very high compression ratio.
 - Excellent visual quality.
 - Efficient storage.
 - Faster transmission.
-

Applications

- JPEG
 - MPEG
 - H.264
 - H.265 (HEVC)
 - AV1
-

4. Lossy Sequential DCT-Based Mode (JPEG)

Introduction

The **Sequential DCT-Based Mode** is the most commonly used JPEG compression method.

It compresses the image **block by block** using the **Discrete Cosine Transform (DCT)**.

Compression Steps

Step 1

Convert RGB image into

- Y (Luminance)
- Cb (Blue Chrominance)
- Cr (Red Chrominance)

Human vision is more sensitive to brightness than color, so chrominance can be compressed more aggressively.

Step 2

Divide image into

8 × 8 Pixel Blocks

Step 3

Apply Discrete Cosine Transform (DCT)

The DCT converts pixel values into frequency coefficients.

Mathematically,

The output contains:

- Low-frequency coefficients (important image information)
 - High-frequency coefficients (fine details and noise)
-

Step 4

Quantization

High-frequency coefficients are divided by larger values and rounded, causing many coefficients to become zero.

This stage achieves the greatest reduction in file size but introduces information loss.

Step 5

Zigzag Scanning

Coefficients are rearranged into a one-dimensional sequence to group zeros together efficiently.

Step 6

Entropy Coding

The resulting sequence is compressed using:

- Run Length Encoding (RLE)
 - Huffman Coding
-

Advantages

- Excellent image quality.
 - Small file size.
 - Widely supported.
 - Efficient for photographs.
-

5. Expanded Lossy DCT-Based Mode (Progressive JPEG)

Definition

Expanded Lossy DCT-Based Mode, commonly implemented as **Progressive JPEG**, transmits the compressed image in multiple scans instead of one continuous scan.

Working Principle

Instead of displaying the image line by line, it first displays a **low-quality preview**, then gradually improves the quality as additional data arrives.

Transmission Process



High Quality Image



Final Image

Advantages

- Faster perceived loading on slow networks.
 - Users see a preview immediately.
 - Better user experience for web applications.
 - Commonly used on websites.
-

Disadvantages

- Slightly more complex encoding and decoding.
 - Requires multiple decoding passes.
-

6. JPEG Compression

Definition

JPEG (Joint Photographic Experts Group) is the world's most widely used image compression standard.

It primarily uses:

- DCT (Discrete Cosine Transform)
 - Quantization
 - Huffman Coding
-

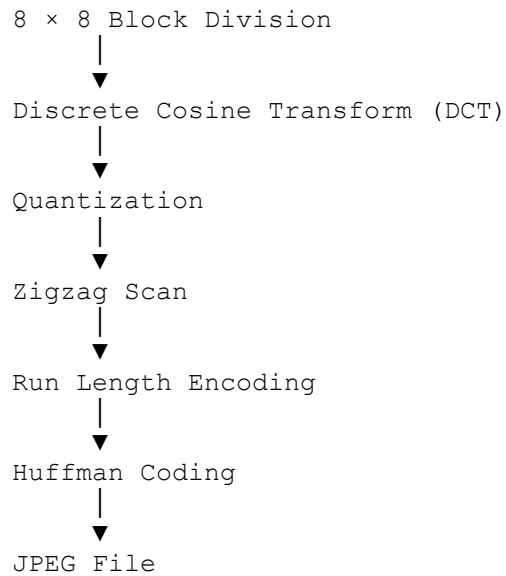
JPEG Compression Pipeline

RGB Image

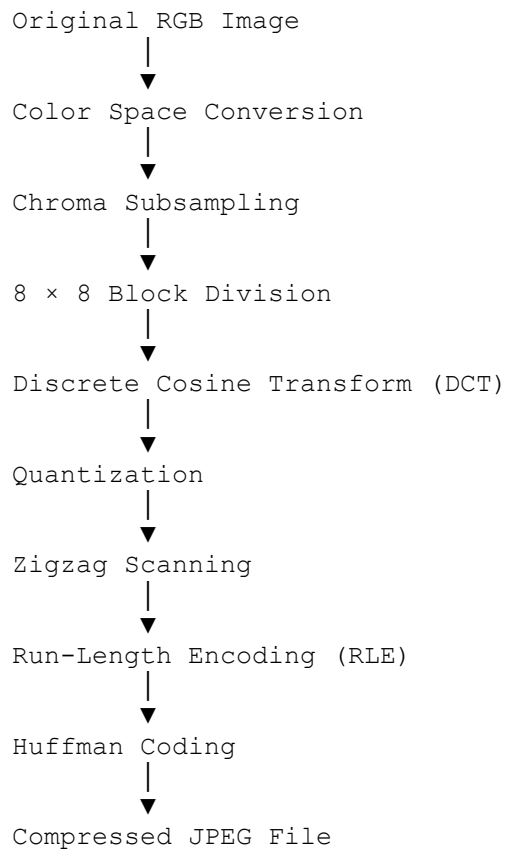


Color Space Conversion





JPEG Compression Flowchart



Step 1: Color Space Conversion

Purpose

Digital cameras capture images in the **RGB (Red, Green, Blue)** color model. JPEG converts RGB into **YCbCr** because the human eye is more sensitive to brightness than to color differences.

Components

- **Y (Luminance):** Brightness information.
- **Cb (Chrominance Blue):** Blue color difference.
- **Cr (Chrominance Red):** Red color difference.

Benefits

- Separates brightness from color.
 - Enables stronger compression of color information.
 - Maintains good visual quality.
-

Step 2: Chroma Subsampling

Purpose

Since the human eye is less sensitive to color than brightness, JPEG reduces the resolution of the chrominance channels.

Common sampling formats include:

- **4:4:4** – No subsampling
- **4:2:2** – Half horizontal color resolution
- **4:2:0** – Half horizontal and vertical color resolution (most common)

Advantages

- Reduces file size.
 - Little visible quality loss.
 - Improves compression efficiency.
-

Step 3: Divide Image into 8×8 Blocks

The image is divided into small blocks of 8×8 pixels.

Example:

```
+-----+-----+
| 8x8   | 8x8   |
+-----+-----+
| 8x8   | 8x8   |
+-----+-----+
```

Each block is compressed independently.

Why 8×8 Blocks?

- Efficient mathematical processing.
- Low computational complexity.
- Good balance between quality and compression.

Step 4: Apply Discrete Cosine Transform (DCT)

Purpose

The **Discrete Cosine Transform (DCT)** converts pixel values from the **spatial domain** into the **frequency domain**.

Instead of representing individual pixel values, DCT represents how image intensity changes within the block.

Mathematical Formula

$$\left[F(u,v) = \frac{1}{4} C(u)C(v) \sum_{x=0}^7 \sum_{y=0}^7 f(x,y) \cos\left(\frac{(2x+1)u\pi}{16}\right) \cos\left(\frac{(2y+1)v\pi}{16}\right) \right]$$

where:

- $(f(x,y))$ = Pixel intensity
- $(F(u,v))$ = DCT coefficient
- $(C(u), C(v))$ = Normalization constants

DCT Output

- **Low-frequency coefficients:** Represent smooth image areas and contain most visual information.
- **High-frequency coefficients:** Represent fine details, edges, and noise.

Step 5: Quantization (Lossy Stage)

Purpose

Quantization reduces the precision of DCT coefficients.

This is the **only lossy step** in JPEG compression.

Process

Each DCT coefficient is divided by a corresponding value in a **quantization table** and rounded to the nearest integer.

$$Q(u,v) = \text{Round} \left(\frac{F(u,v)}{T(u,v)} \right)$$

where:

- $(F(u,v))$ = DCT coefficient
- $(T(u,v))$ = Quantization table value
- $(Q(u,v))$ = Quantized coefficient

Example

Original DCT	Quantization Value	Quantized Value
160	16	10
40	20	2
5	10	1
2	20	0

Many high-frequency coefficients become **0**, greatly reducing the amount of information that must be stored.

Advantages

- Achieves most of the compression.
- Maintains acceptable image quality.

Disadvantages

- Permanent information loss.
- Excessive compression may cause blocking artifacts.

Step 6: Zigzag Scanning

Purpose

The 8×8 matrix of quantized coefficients is converted into a one-dimensional sequence.

Why Zigzag?

The scan starts with low-frequency coefficients and gradually moves toward high-frequency coefficients.

```
1   2   6   7  15 16 28 29
3   5   8  14 17 27 30 43
4   9  13 18 26 31 42 44
10 12 19 25 32 41 45 54
11 20 24 33 40 46 53 55
21 23 34 39 47 52 56 61
22 35 38 48 51 57 60 62
36 37 49 50 58 59 63 64
```

This ordering groups long sequences of zeros together, making subsequent compression more efficient.

Step 7: Run-Length Encoding (RLE)

Purpose

Run-Length Encoding compresses consecutive repeated values, especially zeros produced during quantization.

Example

Original sequence:

15, 8, 3, 0, 0, 0, 0, 0, 2, 0, 0

After RLE:

(15), (8), (3), (0×5), (2), (0×2)

Advantages

- Simple implementation.
- Highly effective after zigzag scanning.

Step 8: Huffman Coding

Purpose

Huffman Coding is the final stage of JPEG compression.

Frequently occurring values receive **short binary codes**, while less frequent values receive **longer binary codes**.

Example

Symbol	Frequency	Huffman Code
0	Very High	0
2	Medium	10
8	Low	110
15	Very Low	111

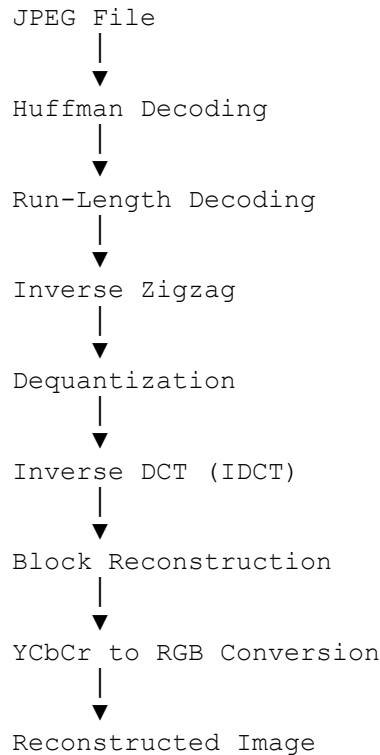
This minimizes the average number of bits required to store the compressed data.

Step 9: JPEG Bitstream Generation

The Huffman-coded data, together with image metadata (headers, quantization tables, and Huffman tables), forms the final **JPEG (.jpg)** file.

JPEG Decompression

To reconstruct the image, the decoder performs the reverse operations:



Since quantization discarded some information, the reconstructed image is **not identical** to the original.

Advantages of JPEG Compression

- High compression ratio (typically 10:1 to 20:1 or higher).
 - Excellent visual quality for photographs.
 - Reduces storage and transmission costs.
 - Universally supported across devices and software.
 - Fast compression and decompression.
-

Advantages

- High compression ratio.
 - Excellent visual quality.
 - Universal support.
 - Fast decoding.
-

Disadvantages

- Information loss.
 - Blocking artifacts at high compression.
 - Repeated editing reduces quality.
-

7. MPEG Compression

Definition

MPEG (Moving Picture Experts Group) is a family of standards for compressing digital video and audio.

Unlike JPEG, MPEG compresses **both spatial and temporal redundancy**.

Types of Frames

I-Frame (Intra Frame)

- Complete image.
 - Similar to JPEG.
 - Can be decoded independently.
-

P-Frame (Predicted Frame)

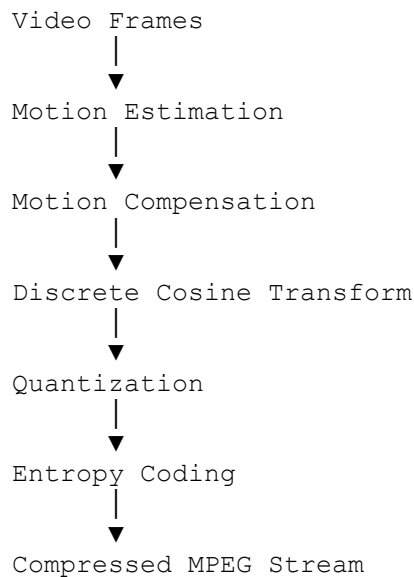
Stores only the changes compared to the previous I-frame or P-frame.

B-Frame (Bidirectional Frame)

Uses information from both previous and future frames.

Provides the highest compression efficiency.

MPEG Compression Process



MPEG Standards

Standard	Application
MPEG-1	Video CD (VCD), early digital video
MPEG-2	DVD, Digital Television (DTV), HDTV
MPEG-4	Internet streaming, mobile video
H.264/AVC	Video conferencing, YouTube, Blu-ray
H.265/HEVC	4K and 8K streaming
AV1	Modern web streaming with high efficiency

Advantages of MPEG

- Excellent video quality.
 - High compression ratio.
 - Reduces storage requirements.
 - Efficient network transmission.
 - Supports high-definition and ultra-high-definition video.
-

Comparison Between JPEG and MPEG

Feature	JPEG	MPEG
Media Type	Still Images	Video and Audio
Compression Type	Spatial Compression	Spatial + Temporal Compression
Main Transform	DCT	DCT with Motion Compensation
Frame-Based	No	Yes (I, P, B Frames)
Compression Efficiency	Moderate	Higher for video
Common Applications	Digital photos, web images	Streaming, television, video conferencing

Summary

Source coding reduces redundancy to improve storage and transmission efficiency. **Entropy coding** is a lossless technique that uses probability-based variable-length codes, such as Huffman and Arithmetic coding, to approach the theoretical minimum number of bits required. **Hybrid coding** combines prediction, transform coding, quantization, and entropy coding to achieve superior compression performance and forms the basis of modern standards such as JPEG and MPEG. JPEG uses **Discrete Cosine Transform (DCT)** for efficient image compression, while MPEG extends these concepts with **motion estimation, motion compensation, and I/P/B frame structures** to compress video efficiently. These techniques are fundamental to modern multimedia systems, enabling high-quality storage, streaming, and communication while minimizing storage and bandwidth requirements.